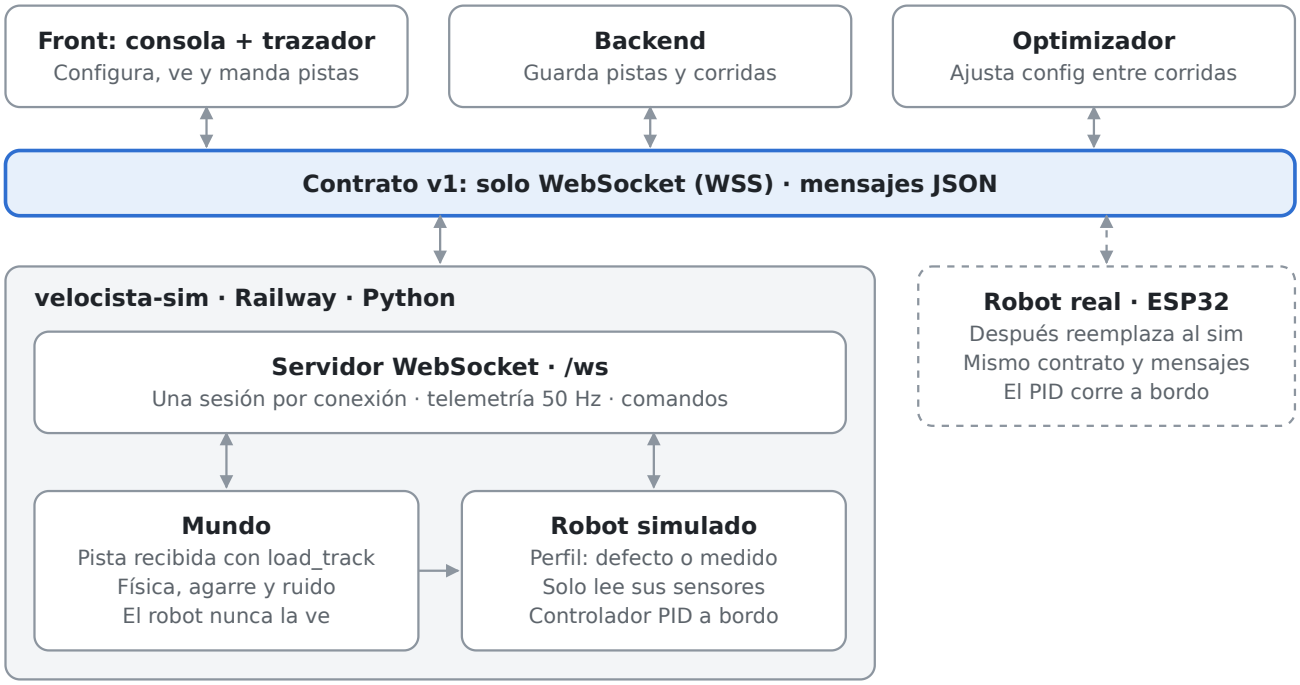
Simulador de velocista — Contrato v1

velocista-sim · Borrador v1.1 · 15 de octubre de 2026

Todos los clientes se conectan **solo por WebSocket** y reciben JSON. Hoy responde el simulador; cuando exista el robot real (ESP32), hablará el mismo contrato y solo cambiará la URL. El PID corre a bordo (en el robot o en el simulador): los clientes solo reciben datos y mandan configuración.

1. Arquitectura inicial



- **Pistas:** se diseñan en el trazador (front), las guarda el backend y llegan al simulador con `load_track` solo cuando se van a usar. Solo las recibe el *mundo*; el robot simulado nunca las ve, igual que el robot real.
- **Optimizador:** es un cliente más del contrato (`set_config` → corre → lee `run_finished`). Falta decidir si va dentro del backend o en un repo aparte.
- **Robot real:** responde `hello`, telemetría y comandos igual que el simulador. Los comandos propios del simulador (`reset`, `load`, `set_profile`, `load_track`) le devuelven `error`.

2. Conexión

```
wss://<proyecto>.up.railway.app/ws?robot=velocista-01&track=oval-01
```

- `robot` y `track` son opcionales: el simulador trae un perfil y una pista de ejemplo para funcionar solo. El robot real ignora ambos.
- Al conectarse, el servidor crea una sesión propia y envía `hello` + `config`. Lo que cambie un cliente no afecta a los demás. La sesión termina al cerrar la conexión.

3. Formato y unidades

JSON UTF-8, **un mensaje por frame** de texto. Todo mensaje lleva `type`, `v` (versión del contrato) y `t_ms` (ms desde el inicio de la sesión). Unidades SI en todo el contrato.

Magnitud	Unidad / rango	Magnitud	Unidad / rango
Tiempo	ms	Sensores de línea	ADC crudo 0–4095 (12 bits)
Distancia, posición	m	`line_position`	0 a (sensores − 1) × 1000; centro 7500 con 16
Velocidad	m/s	PWM motores	−255 a 255
Batería	V	Ángulos	rad

4. Perfil del robot: checklist para armar un robot simulable

Cada dato tiene un **valor por defecto** (datasheet o estimado) y un **valor medido** que lo reemplaza cuando se mide el robot real. Todos se pueden modificar. El simulador usa el medido si existe; si no, el de defecto.

```
"wheelbase_m": {"default": 0.130, "measured": null, "source": "estimado"}
```

Dato	En `hello`	Por defecto (v1)	Origen del defecto
ID, firmware, revisión de hardware	Sí	`velocista-01`, `sim-0.1`, `A`	Definido
Sensores de línea · bits del ADC	Sí	16 · 12 bits	Robot actual (2× QTR-8A + mux)
Separación entre sensores	No	9,525 mm	Datasheet QTR-8A
Distancia de la regleta delante del eje	No	85 mm	Estimado
Lectura en blanco / negro · ruido	No	≈200 / ≈3900 ADC · ±60	Estimado (medible ya con la regleta)
Cantidad de motores	Sí	2	Robot actual
Velocidad máxima a PWM 255 · respuesta del motor	No	2,8 m/s · τ = 45 ms	Estimado
Distancia entre ruedas	No	130 mm	Estimado
Agarre lateral máximo	No	9 m/s² (≈0,9 g, sin turbina)	Estimado
Turbina · succión	Sí · No	No tiene	Robot actual
Encoders · ticks por vuelta	Sí · No	No tiene	Robot actual
IMU	Sí	No tiene	Robot actual
Batería: celdas · voltaje nominal	Sí	2S · 7,4 V	Estimado
Periodo del loop de control	No	2 ms (500 Hz)	Estimado

En `hello` va lo que cambia **qué datos llegan**, porque los clientes lo necesitan para interpretar la telemetría. Lo físico solo lo usa el simulador para comportarse como el robot. La lista completa de componentes con precios y consumo nominal vive en el catálogo del framework, enlazada por ID y revisión de hardware.

5. Mensajes que se reciben y frecuencia

Mensaje	Cuándo llega
`hello`	Una vez, al conectarse
`config`	Al conectarse y tras cada cambio de configuración
`telemetry`	**Cada 20 ms (50 Hz)** mientras la conexión esté abierta; ajustable de 10 a 100 Hz con `set_rate`
`event`	Al instante: `run_started`, `lap`, `off_track`, `run_finished`
`catalog`	Como respuesta a `list`
`error`	Cuando un comando es inválido o no aplica (p. ej. `load_track` al robot real)

Una **corrida** (`run_id`) va de `start` hasta `stop` o una salida de pista; sirve para agrupar datos al guardarlos.

hello — describe el robot; el resto de mensajes se interpreta según esto

```
{"type":"hello","v":1,"t_ms":0,"session_id":"s-7f3a","source":"sim",
 "robot":{"id":"velocista-01","firmware":"sim-0.1","hw_rev":"A"},
 "capabilities":{"line_sensors":16,"adc_bits":12,"motors":2,
                 "turbine":false,"encoders":false,"imu":false,
                 "battery":{"cells":2,"nominal_v":7.4}}}
```

`source` es `"sim"` o `"robot"`. Así cada cliente sabe si llegará `sim_truth`.

telemetry — robot actual (sin encoders ni IMU)

```
{"type":"telemetry","v":1,"t_ms":30640,"seq":1532,"run_id":"r-0004","running":true,
 "sensors":[210,198,3870,3912,640,205,201,199,203,197,200,204,198,202,199,201],
 "line_position":7342,"line_lost":false,"error":-158,
 "control":{"p":-7.1,"i":0,"d":2.3,"out":-4.8},
 "motors":{"left":172,"right":128},
 "battery_v":7.91,
 "lap":{"n":3,"current_ms":1210},
 "sim_truth":{"x":0.52,"y":-0.31,"theta":1.57,"speed_mps":1.64,"distance_m":12.4}}
```

- `sensors` tiene tantos valores como `capabilities.line_sensors`. `seq` es consecutivo: si salta, se perdió un mensaje. `motors` es el PWM que se aplica en ese instante.
- `battery_v` solo llega si el robot mide su voltaje (p. ej. con un divisor a un pin ADC). Si no lo mide, no se envía.
- **No hay velocidad ni distancia medidas:** sin encoders el robot no puede saberlas. Lo único medible es el tiempo de vuelta.
- `sim_truth` es la verdad exacta del simulador (posición y velocidad reales). **Solo existe si `source` es `"sim"`**. Sirve para depurar y comparar, pero nada que deba funcionar con el robot real puede depender de ella.

event

```
{"type":"event","event":"run_started","run_id":"r-0004","t_ms":27200}
{"type":"event","event":"lap","run_id":"r-0004","t_ms":30640,"lap":3,"lap_ms":3416}
{"type":"event","event":"off_track","run_id":"r-0004","t_ms":41200,"reason":"line_lost"}
{"type":"event","event":"run_finished","run_id":"r-0004","t_ms":41200,
 "summary":{"laps":5,"best_lap_ms":3416,"avg_lap_ms":3480,"off_tracks":1,"track_id":"oval-01",
            "config":{"controller":"pid","kp":0.045,"ki":0,"kd":0.9,"base_speed":180}}}
```

config, catalog y error

```
{"type":"config","v":1,"t_ms":120,"controller":"pid","params":{"kp":0.045,"ki":0,"kd":0.9},
 "base_speed":180,"rate_hz":50,"track_id":"oval-01"}
{"type":"catalog","v":1,"t_ms":900,"robots":["velocista-01"],"tracks":["oval-01"]}
{"type":"error","v":1,"t_ms":5310,"message":"load_track no aplica al robot real"}
```

Ningún mensaje que sale del simulador trae la geometría de la pista, solo su `track_id`.

6. Comandos (cliente → servidor)

Solo configuración y órdenes de corrida, nunca acciones en tiempo real.

Comando	Qué hace	Robot real	Ejemplo
`start`	Inicia corrida (nuevo `run_id`)	Sí	`{"cmd":"start"}`
`stop`	Detiene y emite `run_finished`	Sí	`{"cmd":"stop"}`
`set_config`	Controlador, parámetros, velocidad base	Sí	`{"cmd":"set_config","controller":"pid","params":{"kp":0.045,"ki":0,"kd":0.9},"base_speed":180}`
`set_rate`	Frecuencia de telemetría (10–100 Hz)	Sí	`{"cmd":"set_rate","hz":20}`
`list`	Robots y pistas disponibles	Sí*	`{"cmd":"list"}`
`reset`	Vuelve el robot a la línea de salida	No	`{"cmd":"reset"}`
`load`	Cambia el perfil de robot	No	`{"cmd":"load","robot_id":"velocista-01"}`
`set_profile`	Cambia valores del perfil (p. ej. uno medido)	No	`{"cmd":"set_profile","measured":{"wheelbase_m":0.128}}`
`load_track`	Carga una pista completa en el mundo	No	`{"cmd":"load_track","track":{…formato de la sección 7…}}`

* El robot real responde solo consigo mismo y sin pistas.

7. Formato de pista (track v1)

El trazador ya guarda el diseño en JSON (vértices con radio, cortes, zonas y salida, en cm) y su función `build()` ya calcula la línea central muestreada cada 0,5 cm. **Cambio pendiente en el trazador:** agregar la exportación "Para el simulador", que use `build()` y genere este formato. Así la geometría se calcula en un solo lugar.

```
{"id":"junior-02","v":1,"name":"Técnica 3 × 2,5","units":"m",
 "board":{"width":3.00,"height":2.50},
 "line_width":0.019,"closed":true,
 "centerline":[[0.500,0.450],[0.505,0.450], …],
 "gaps":[{"at":4.82,"length":0.02}],
 "start":{"at":0.70},
 "rules_profile":"lgr2026"}
```

`centerline` son puntos en metros, cada 5 mm. `at` es la distancia en metros a lo largo de la línea desde el primer punto. `rules_profile` es el perfil de reglas del trazador (reglamento), no el perfil del robot. Las marcas laterales no existen hoy en el trazador: quedan como campo opcional (`marks`) para una versión posterior.

8. Encoders e IMU: previstos, no implementados en v1

- **Contrato v1:** ya los contempla. `hello` los declara y, si están, la telemetría agrega un bloque `odometry`. Agregarlos no rompe a ningún cliente.
- **Simulador v1:** no los simula. Encoders e IMU simulados (con ruido y deriva) van en la **v2 del simulador**, cuando se decida montarlos en el robot. Hoy no hay hardware para validar esos modelos, y la prioridad del sprint 1 es que el equipo pueda trabajar.

```
"odometry":{"distance_m":12.4,"speed_mps":1.62,"heading_rad":1.55}   // solo si encoders o imu = true
```

Encoders → distancia y velocidad. Giroscopio de la IMU → giro y curvatura. La IMU sola no sirve para velocidad ni distancia (la deriva crece en segundos).

9. Notas para backend y optimizador

- Cada `telemetry` pesa ~0,5 KB → ~25 KB/s por cliente a 50 Hz. No guardar cada frame: eventos y `run_finished` siempre; telemetría submuestreada o comprimida por `run_id`.
- El optimizador trabaja con `run_finished.summary`. Hoy solo puede optimizar contra el tiempo de vuelta y las salidas de pista, que es lo que el robot real también puede medir.
- **Versionado:** agregar campos no rompe nada; renombrar o quitar uno sube `v`. El contrato formal (AsyncAPI + JSON Schema) vivirá en el repo `velocista-sim`; los cambios se proponen por PR.